5G SIP Voice Testbed - Technical Report

1. Introduction

This report documents the implementation and testing of SIP-based voice services running on top of a 5G standalone (SA) data plane. The testbed demonstrates end-to-end Voice over IP (VoIP) calls where SIP signaling and RTP media traffic traverse a simulated 5G network, proving that traditional telephony services can be delivered over 5G user-plane infrastructure.

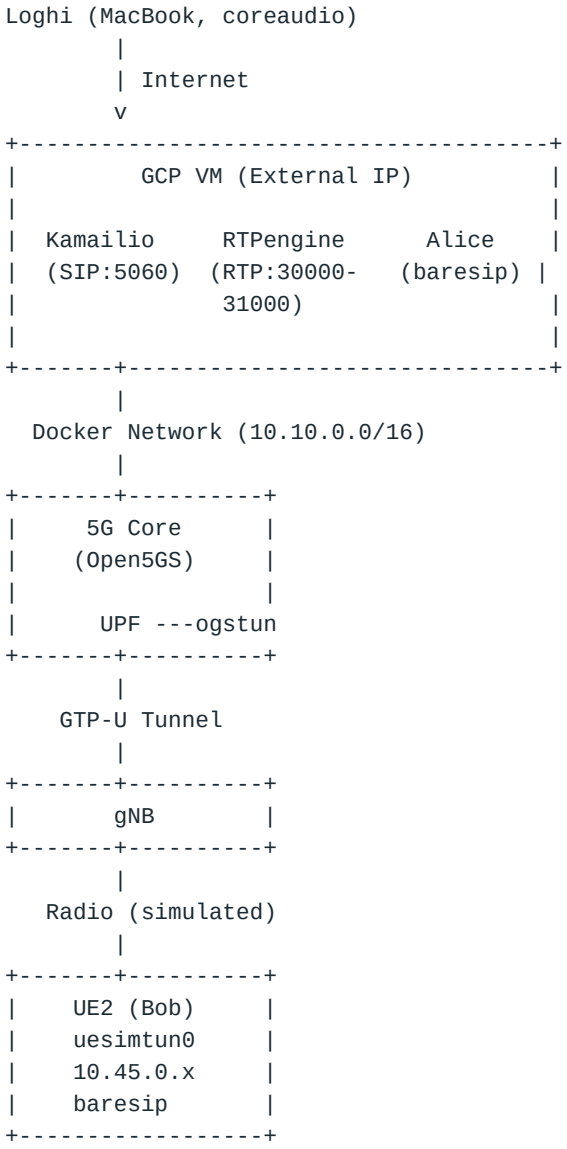
Repository: https://github.com/loghijiaha/5g-sip-testbed

2. System Architecture

The testbed consists of the following components deployed on a Google Cloud Platform (GCP) VM:

Component	Role	Deployment
Open5GS	5G Core Network (AMF, SMF, UPF, NRF, AUSF, UDM, UDR, PCF, BSF, NSSF)	Docker containers
UERANSIM	5G RAN simulator (gNB + UE)	Docker containers
Kamailio 6.0.1	SIP Proxy/Registrar	Native on VM
RTPengine	RTP Media Relay	Docker container
baresip	SIP User Agents (Alice, Bob, Loghi)	VM native + Docker + Mac

2.1 Network Topology



3. Test Call Procedure

3.1 Participants

• **Loghi**: External SIP client on MacBook (real microphone via coreaudio), connected over the internet to the GCP VM's public IP.

• **Alice**: SIP client running natively on the GCP VM (file-based audio source).

• **Bob**: SIP client inside the UE2 Docker container, with all traffic routed through the 5G tunnel interface (uesimtun0).

3.2 Call Setup Steps

1. **Start 5G Core**: All Open5GS NFs launched via Docker Compose. Subscribers provisioned in MongoDB.

2. **UE Registration**: UE2 registers with the 5G core via UERANSIM. PDU session established, uesimtun0 interface created with IP 10.45.0.x.

3. **SIP Registration**: Bob's baresip registers with Kamailio through the 5G tunnel (uesimtun0 → GTP → UPF → VM).

4. **Call Initiation**: Loghi (Mac) dials Bob via Kamailio. SIP INVITE traverses: Mac → Internet → Kamailio → 5G tunnel → Bob.

5. **Call Answer**: Bob auto-answers with 200 OK. RTPengine allocates media relay ports.

6. **Media Flow**: RTP audio from Loghi's microphone → RTPengine → 5G tunnel (ogstun) → Bob. Bob's audio (WAV file) → 5G tunnel → RTPengine → Loghi.

7. **Call Termination**: BYE sent through Kamailio, properly tearing down the session.

3.3 Key Configuration

• Bob's `net_interface` set to `uesimtun0` — forces all SIP/RTP to traverse the 5G data plane.

• IP route `dev uesimtun0` added in UE2 — ensures traffic to Kamailio hairpins through the 5G core.

• RTPengine configured with `--interface=10.10.0.30!` — advertises external IP to remote clients while listening on Docker network.

4. Verification Results

4.1 5G Data Plane Verification

```
$ docker exec ueransim-ue2 ping -I uesimtun0 -c 3 8.8.8.8
64 bytes from 8.8.8.8: icmp_seq=1 ttl=59 time=1.80 ms
```

4.2 SIP Registration Through 5G (ogstun capture)

```
$ docker exec open5gs-upf tcpdump -i ogstun -n port 5060 -c 4
10.45.0.2.5080 > 136.111.254.162.5060: SIP: REGISTER sip:136.111.254.162
136.111.254.162.5060 > 10.45.0.2.5080: SIP: SIP/2.0 200 OK
```

Bob's SIP REGISTER traverses ogstun → confirms SIP signaling over 5G user plane.

4.3 SIP Call Signaling Through 5G

```
$ docker exec open5gs-upf tcpdump -i ogstun -n udp -c 10
10.45.0.2.5080 > 136.111.254.162.5060: SIP: INVITE sip:loghi@...
136.111.254.162.5060 > 10.45.0.2.5080: SIP: SIP/2.0 100 trying
136.111.254.162.5060 > 10.45.0.2.5080: SIP: SIP/2.0 180 Ringing
136.111.254.162.5060 > 10.45.0.2.5080: SIP: SIP/2.0 200 Answering
10.45.0.2.5080 > 136.111.254.162.5060: SIP: ACK sip:loghi@...
```

Complete SIP dialog (INVITE → 100 → 180 → 200 → ACK) traverses the 5G data plane.

4.4 RTP Media Through 5G (ogstun capture)

```
$ docker exec open5gs-upf tcpdump -i ogstun -n udp and not port 5060 -c 10
10.10.0.30.30332 > 10.45.0.2.30304: UDP, length 172
10.10.0.30.30332 > 10.45.0.2.30304: UDP, length 172
10.10.0.30.30332 > 10.45.0.2.30304: UDP, length 172
...
```

172-byte UDP packets (12-byte RTP header + 160-byte PCMU payload = 20ms audio frame) flow from RTPengine through ogstun to Bob's UE. This proves RTP audio media traverses the 5G user plane.

4.5 Kamailio SIP Logs

```
REGISTER sip:bob@136.111.254.162      [OK]
REGISTER sip:loghi@101.127.123.149   [OK]
INVITE   sip:loghi@136.111.254.162 -> sip:bob@136.111.254.162
ACK      sip:loghi@101.127.123.149 -> sip:bob@136.111.254.162
```

```
BYE      sip:bob@136.111.254.162   -> sip:loghi@101.127.123.149
```

4.6 Call Establishment (baresip output)

Loghi (Mac):

```
loghi@136.111.254.162: Call established: sip:bob@136.111.254.162
[0:01:44] audio=63978/0 (bit/s)
```

Bob (UE2 via 5G):

```
bob@136.111.254.162: Call established: sip:loghi@136.111.254.162
stream: incoming rtp for 'audio' established, receiving from 10.10.0.30:30332
audio tx pipeline: aufile ---> PCMU
[0:00:30] audio=63978/0 (bit/s)
```

5. Conclusion

This testbed successfully demonstrates SIP-based voice services operating on top of a 5G SA data plane:

1. **SIP signaling** (REGISTER, INVITE, ACK, BYE) traverses the 5G user plane via the UPF's ogstun interface.
2. **RTP media** (PCMU audio at 64 kbit/s) is relayed through RTPengine and delivered to the UE via the GTP tunnel.
3. **End-to-end calls** between an external client (MacBook with real microphone) and a simulated UE inside the 5G network are established with proper call setup and teardown.
4. **The 5G data plane** (Open5GS UPF) correctly handles both SIP signaling and RTP media packets, demonstrating that voice services can be layered on top of 5G without requiring IMS.

The complete testbed is reproducible using Docker Compose and the scripts in the repository.